

CALIPER

Empirical Measurement of Container Confinement

Project Proposal
B.Tech, Computer Science and Engineering, Final Year
Four-member team

1. OVERVIEW

A container's security posture is declared in a manifest and enforced by the kernel. These are not the same thing. The manifest names a seccomp profile, a capability set and a Pod Security Standard; what the kernel actually permits depends additionally on the container runtime, its version, the node's distribution and loaded modules, the kernel build, the processor architecture, the active Linux Security Module and the RuntimeClass. Two clusters running an identical manifest can expose materially different kernel attack surface, and nothing in the deployment pipeline reports the difference.

Existing tooling addresses the two ends of this problem and not the middle. Configuration scanners read the manifest and report what was declared. Runtime monitors observe what the workload did. Neither answers the question that sits between them: given this container, on this node, right now, what can it actually reach in the kernel?

CALIPER is an instrument that answers that question by measurement. It runs as an ordinary container, executes a corpus of argument-level reachability probes against the kernel, records the exact verdict returned for each, and emits a machine-readable confinement fingerprint keyed by the environment that produced it. Fingerprints are diffed against one another, against the declared manifest, and against a maintained reference posture, and the divergences are reported with the minimal configuration change that closes them.

2. MOTIVATION

The failure mode is documented and recent. CVE-2026-31431, tracked as Copy Fail, is a logic flaw in the algif_aead module reached through the AF_ALG address family. Combined with splice(2) it writes four bytes into the page cache of any readable file, which is sufficient to modify a setuid binary in memory without altering it on disk. CISA added it to the Known Exploited Vulnerabilities catalogue on 1 May 2026.

Three properties of that vulnerability motivate this project.

First, it is reached through a socket address family, not through a distinctive syscall. Filtering at syscall-name granularity cannot express the distinction between a container that may call socket(2) and one that may reach the kernel crypto API, and syscall-name granularity is what every profile generator on the market emits.

Second, published testing across Talos and EKS found that the RuntimeDefault seccomp profile in both containerd and Moby did not deny AF_ALG, and that Pod Security Standards at the Restricted level did not prevent it either. Workloads that satisfied every conventional hardening check remained exploitable, and establishing this required hand-built test images and a hand-authored profile.

Third, exploitability depends on whether algif_aead is loaded on the node, which is a property of the distribution's module policy and not of anything the workload author controls or can observe. The same manifest is exposed on one node and not on another.

The general problem is that RuntimeDefault is runtime-defined. Its contents are a property of the runtime binary on the node, not of the manifest, and they change over time. io_uring, which defeats syscall filtering entirely by performing operations without issuing the corresponding syscalls, was removed from the default profile of Moby and containerd in November 2023, and the change shipped in Moby 25.0 in January 2024 and in containerd 2.0 in November 2024 without a backport to containerd 1.7. AF_ALG itself followed the same path in May 2026, when Moby added a deny for it in 29.4.2. A cluster whose node image predates those releases enforces a weaker policy than an identically configured cluster whose node image does not, and neither manifest records the distinction.

The declared posture is therefore an assertion, and it is currently unverified. CALIPER exists to verify it.

3. RELATION TO EXISTING WORK

Work in this area falls into three categories, none of which measures enforced confinement.

Configuration scanners, including kube-bench, Kubescape and Trivy, parse manifests, Helm charts and cluster configuration against benchmarks. They report what was declared. In the AF_ALG case the declaration was correct and the enforcement was weaker than practitioners believed, and a configuration scanner reports no finding.

Runtime monitors, including Falco, Tetragon and commercial equivalents, observe syscalls as the workload issues them. They report what the workload did, which is a subset of what it could have done. An attack surface that is never exercised is invisible to them.

Profile generators, including oci-seccomp-bpf-hook, the Kubernetes Security Profiles Operator, Inspektor Gadget and slimtoolkit, trace a workload and emit a policy. They answer what the application needs, not what the environment permits, and they emit policy at syscall-name granularity, which is the granularity at which the AF_ALG case is invisible.

Two tools are direct ancestors. amicontained reports the container runtime, namespaces, capabilities and a coarse list of blocked syscalls; it established the idea, and its last release was v0.4.9 in November 2019 with

its last commit in September 2020. seccomp-diff extracts and disassembles the seccomp-BPF filter from a running process and diffs two of them; it is confined to seccomp, requires privilege that an unprivileged container does not hold, and reasons about the filter rather than about reachability, so it cannot account for capabilities, LSM policy, mount masking, loaded modules or architecture.

Houdini (Patel et al., PST 2025) proposes benchmarking container security confinement and is the closest published statement of the problem. It is a research contribution and no tool was released.

CALIPER is an engineering contribution and is presented as such. Its claim is not that the idea is new but that no instrument implementing it exists.

4. OBJECTIVES

1. To implement a probe engine that executes reachability probes under fork isolation, with bounded and accounted side effects, and classifies each outcome precisely, distinguishing permitted, denied, unimplemented and fatally filtered results.
2. To implement a probe corpus operating at argument granularity, covering, as the committed set, socket address families and netlink protocol numbers, io_uring opcode reach, mount filesystem types, clone and unshare flags, masked and read-only path coverage, device node access, and capability effect. A deferred set, comprising ioctl request codes, bpf commands and program types, keyctl operations, prctl options, perf_event_open, userfaultfd, open_by_handle_at, module loading and seccomp user notification, is specified in the same format and implemented as time permits.
3. To implement denial attribution based principally on errno oracles, identifying which enforcement mechanism produced a given denial, and reporting attribution confidence where the mechanism cannot be determined without privilege.
4. To define a stable confinement fingerprint format whose identity includes processor architecture, kernel version, distribution and loaded modules, runtime and runtime version, LSM and RuntimeClass, and a diff engine that reports divergence at probe granularity and classifies every divergence as architecture-explained, kernel-version-explained, runtime-class-explained or policy-explained.
5. To author and maintain a reference posture: a versioned, cited model asserting which kernel entry families a workload at a named security tier should not reach, against which measurements are evaluated.
6. To implement a reporting layer that evaluates a fingerprint against both the declared manifest and the reference posture, and

distinguishes claim violations, expectation violations and undeclared exposure.

7. To implement a remediation generator that emits the minimal argument-level policy closing the divergences found, expressed both as a Localhost seccomp profile and as a Security Profiles Operator custom resource, and verifies closure by re-probing.
8. To evaluate the instrument across at least six distinct environment cells spanning architecture, runtime, runtime version, LSM and RuntimeClass, reporting measured divergence and reproducing the AF_ALG case unaided.

5. ARCHITECTURE

Probe Engine	fork isolation, probe execution, verdict classification, side effect accounting
Probe Corpus	argument-level probe definitions with their attribution metadata, grouped by entry family
Attributor	errno oracles, capability and LSM correlation, confidence scoring
Fingerprint and Diff	environment-keyed serialisation, divergence reporting, divergence classification
Evaluator	manifest parsing, reference posture evaluation, finding categorisation
Remediator	minimal policy emission, closure verification

The fingerprint format is the fixed interface of the system. Its first version is agreed in week two and developed against thereafter, and it is frozen at the end of week nine, after which any change is a new format version. The engine and corpus produce the measurement, the control plane completes it into a fingerprint, and the diff engine, evaluator and remediator consume it. This allows the four workstreams to proceed without blocking one another, and allows the reporting layers to be developed against recorded fingerprints long before the corpus is complete.

The fingerprint's identity is the environment cell, not the workload. Architecture, kernel version, distribution and loaded modules, runtime and runtime version, LSM and RuntimeClass are recorded as first-class fields, because a divergence between an arm64 and an amd64 measurement is not a policy finding and must not be reported as one. The loaded module set is recorded before any probe executes, for the reason given in 6.2.

The cell is assembled by the control plane, not by the probe. The probe records what is observable from inside an unprivileged container: the architecture, the kernel's own version string, the loaded module set, its capability sets and security-module label, and the seccomp mode. Runtime,

runtime version, distribution and RuntimeClass are not observable from inside, and are taken from the Kubernetes Node object and the pod's RuntimeClass, or supplied by the operator for a run outside Kubernetes, and each is recorded together with its source. Attribution is computed by the control plane from the raw verdicts and the probe's declared metadata; the engine stores no interpretation. A kernel version reported by a sandbox, as gVisor reports a synthetic one, is recorded as the sandbox's claim and the RuntimeClass field, not the version, drives classification.

RuntimeClass is handled in the same spirit. gVisor implements a subset of the syscall interface and reports a synthetic kernel version, and Kata boots a guest kernel of its own, so a verdict that differs between a runc cell and a sandboxed cell is explained by the sandbox before any policy is consulted. The classifier therefore carries a runtime-class-explained class, assigned when the two cells differ in RuntimeClass and the sandboxed side reports the entry point unimplemented; it is reported beside the architecture class and never counted as policy. A difference in LSM or in runtime version, by contrast, is a difference in the policy the kernel actually enforces, and is classified as policy-explained.

The probe engine is implemented in Rust. The engine must fork before each risky probe, must not issue syscalls of its own that pollute the measurement, and must survive a probe terminated by SECCOMP_RET_KILL_PROCESS without losing the run. A managed runtime that spawns threads and issues background syscalls is unsuitable on all three counts. The control plane, comprising fingerprint handling, diffing, evaluation, remediation and Kubernetes and CI integration, is implemented in Go, where the container ecosystem libraries are mature. The two halves meet at the fingerprint format and nowhere else.

6. THE TWO HARD PROBLEMS

6.1 Attribution

A probe attempts an operation and receives EPERM. The kernel does not say why. The denial may have come from the seccomp filter, from a dropped capability, from AppArmor or SELinux, from an unmapped identity in a user namespace, or from a read-only or masked mount. From inside the container these are indistinguishable, and a report that cannot tell them apart cannot produce a remediation, because the fix differs in each case.

The primary technique is the errno oracle. Each probe is authored with an argument set chosen so that, if the syscall reaches the kernel at all, the kernel is structurally guaranteed to return a specific and distinctive error: EBADF for an operation on a deliberately invalid descriptor, EINVAL for a malformed request, EAFNOSUPPORT for an address family outside the kernel's range. Receiving EPERM where EBADF was guaranteed proves the call was intercepted before the point at which the kernel would have produced that error, and therefore that a filter and not a capability produced the denial. The technique requires no privilege, works from inside any container, and yields a positive proof rather than an inference. Its reach is per syscall, and each oracle records it: descriptor resolution precedes

every security-module hook, so an EBADF oracle isolates seccomp, whereas the hook for socket(2) runs before the family lookup, so an EAFNOSUPPORT oracle on an in-range family separates a filter from a capability but not from AppArmor or SELinux, which the label interfaces then decide. A filter may also return ENOSYS rather than EPERM, as runc does for the syscalls it stubs, and ENOSYS is treated as ambiguous rather than as unimplemented whenever the cell's kernel is known to implement the syscall. Authoring the oracle is part of authoring the probe, and the corpus is not considered complete until every probe carries one.

Capability attribution is by correlation. The operation's documented capability requirement is recorded with the probe and checked against the effective and bounding sets read from the process's own status, which is readable without privilege. Where a capability is absent and the errno oracle indicates the call reached the kernel, the capability is the cause.

Security-module correlation is the third technique. The process's own AppArmor or SELinux label is readable without privilege from its attr directory under /proc, as is whether the module is enforcing. A denial that the oracle places after the security-module hook, arriving as EACCES under a confining label, is attributed to the module at reduced confidence; under an unconfined label it is not, and the seccomp reading stands.

Filter introspection is a fourth technique and its availability is limited. PTRACE_SECCOMP_GET_FILTER requires CAP_SYS_ADMIN and a ptrace-attachable target, which an unprivileged container does not have, and /proc/self/status exposes only the seccomp mode and a count of installed filters, not their contents. It is therefore available only to a privileged helper, which this proposal outlines but does not specify, and which is not a committed deliverable within the schedule. The design does not depend on it, and the two techniques above need no privilege.

Some denials consequently remain undecidable from inside the container. The design accepts this rather than concealing it. Attribution is reported with a confidence level, and the residual undecidable fraction is reported as a measured quantity rather than assumed to be zero. Reducing that fraction through a privileged helper is the first extension the design admits.

6.2 Probe safety and survivability

An instrument that damages what it measures is not usable in the environments where it is most needed. Every probe must be safe to run against a production node.

Three constraints follow. Probes establish the reachability of a kernel entry point and never its exploitation; the corpus contains no exploit code and no operation whose success confers privilege. Where a subsystem offers an enumeration interface it is preferred over execution: io_uring opcode reach is measured with IORING_REGISTER_PROBE, which reports the supported opcodes without submitting a single queue entry. Every probe declares its side effects and its rollback, and probes with unbounded duration execute under a timeout, with a probe that hangs recorded as such rather than

stalling the run.

Fork isolation is the rollback for most of the corpus. Every side effect that is descriptor-shaped, which includes `io_uring` rings, BPF program and map descriptors, sockets, `userfaultfd` and `perf` events, is reclaimed by the kernel when the probe's child exits. Residual state is therefore measured only for the classes that outlive a process, and is measured directly rather than by whole-host comparison, which normal system churn would swamp: the mount table, the session and user keyrings, `cgroup` entries, System V and POSIX IPC objects, and network namespaces.

One side effect is neither descriptor-shaped nor reclaimable, and is declared rather than rolled back. A `socket(2)` call on an address family the kernel has not registered asks the kernel to load the module for it, as does an unregistered netlink protocol number or a `bind(2)` to an algorithm type on an `AF_ALG` socket, and the kernel does so with no capability check on the caller. The probe that tests whether `algif_aead` is reachable can therefore be the reason it is loaded, on a node that permits autoloading. The engine records the loaded module set before any probe executes, so that the environment cell reflects the node and not the instrument, records it again after the run, and reports the difference as a finding in its own right. The families that can trigger a load declare it, and the reduced corpus of section 12 either omits them or runs them only when the operator asks.

Survivability is the corresponding requirement. A probe that trips a filter whose action is `SECCOMP_RET_KILL_PROCESS` terminates the process that issued it, and an instrument that dies on its first genuine finding is worthless. Each risky probe therefore executes in a forked child whose exit status, including termination by `SIGSYS`, is itself the measurement. The common case does not require this, since the default action of the `RuntimeDefault` profile is `SCMP_ACT_ERRNO` rather than a kill; it is required for the hand-authored hardened profiles that this instrument is most useful against, and it is established in the first implementation phase because every subsequent probe family depends on it.

7. THE REFERENCE POSTURE

The motivating example is not a claim violation. Pod Security Standards at the Restricted level assert nothing about `AF_ALG`, and `RuntimeDefault`'s entire contract is that it contains whatever the runtime ships. Measured against the manifest alone, the `AF_ALG` case produces no finding: the declaration and the measurement agree, and both are weaker than practitioners believe. A checker that compares measurement against the manifest and nothing else would have missed the case this project exists to detect.

What failed was an expectation, not a claim. Detecting it requires a model of what a hardened workload ought not to reach, authored deliberately and independently of what any specification happens to promise.

The reference posture is that model. It is a versioned document that, for

each named security tier, asserts which kernel entry families a workload at that tier should not reach: for an untrusted multi-tenant tier, the kernel crypto API, module loading, perf_event_open, raw netlink, io_uring, the 32-bit ABI, unmasked procfs, and the rest. Every assertion carries a rationale and a citation, whether a CVE, a documented escape technique or a vendor advisory, so that the posture is auditable rather than assertion by authority. It is deliberately stronger than the tiers it is named after, and that gap is the point of it.

The evaluator therefore produces three categories of finding.

Expectation violation	reachable, and the reference posture holds that it should not be. The AF_ALG case.
Claim violation	the manifest asserts a protection that the measurement does not confirm.
Undeclared exposure	reachable, and addressed by neither model. An input to corpus and posture development.

The reference posture also supplies the target that remediation is minimal with respect to. Without it, a minimal patch has nothing to be minimal against. It is released with the instrument and is, in practical terms, the most reusable artefact the project produces.

8. WORK ALLOCATION

Four workstreams, cut along the fingerprint interface so that members are not blocked on one another.

Member A	Probe engine, safety and the attribution engine. Fork isolation, timeout handling, side effect declaration and accounting, verdict classification, and the probe specification format, including the specification of the deferred set. Delivers the engine in the first phase, then owns probe safety for the life of the project and takes half the corpus families. Also builds the errno oracle attribution engine: establishing that a call was intercepted before the kernel reached the point at which the oracle guaranteed its error is a property of the fork-and-verdict path, and belongs with the code that implements it rather than with the corpus that declares the oracles.
Member B	Corpus, oracle discipline and reference posture. The remaining corpus families; the errno oracle discipline across the whole corpus, meaning the design of the first families' oracles, review of every oracle A authors with A's families, and the completeness check that gates the freeze; capability and security-module correlation; the confidence model and the measurement of the undecidable fraction; and the reference posture, whose assertions and

citations follow the same entry families as the probes. Attribution is split rather than owned: B decides what a given errno proves, A builds the machinery that proves it.

Member C Fingerprint, diff and evaluation. Environment-keyed serialisation and versioning, the diff engine and its divergence classifier, manifest parsing, the evaluator, and reporting in JSON and human-readable form, with SARIF as an extension.

Member D Delivery and evaluation harness. Kubernetes Job and DaemonSet packaging, CI integration, the environment matrix and its automation, and the remediation generator with closure verification. Also acts as integrator, responsible for the components composing correctly.

The corpus is split between A and B rather than owned by one member, because it is the largest single body of work and because attribution metadata is authored with each probe rather than added afterwards. The reference posture sits with B rather than C for the same reason: its assertions are the entry families, and the citations are gathered while the corresponding probes are written.

9. IMPLEMENTATION PHASES

The schedule is seventeen weeks. It is compressed relative to a conventional two-semester plan by reducing scope, not by shortening the critical path of the work that remains, and the reductions are stated in 9.2 so that what has been given up is visible rather than discovered later.

9.1 Schedule

Weeks 1-3 Probe engine. Fork isolation, SIGSYS survival, timeout handling, verdict classification, and the probe specification format including its errno oracle, capability requirement and side effect fields. A reference corpus of ten probes running end to end.

In parallel, and against the specification format rather than the engine: fingerprint format design, environment harness scaffolding, and errno oracle design for the first corpus families. No member is idle in this phase.

Weeks 4-8 Corpus construction, split between two members, in the committed order: socket address families and netlink protocol numbers first, then `io_uring` opcode reach, mount filesystem types, clone and unshare flags, masked and read-only paths, device nodes and capability effect. Each probe is authored complete with its errno oracle, capability requirement and side effect declaration. The corpus is frozen at the end of

week 8; the deferred set is entered only if the committed set is complete and the schedule is intact.

Weeks 4-8	Reference posture, in parallel. Tier definitions, assertions and citations, developed against the corpus as it lands.
Weeks 4-9	Fingerprint and diff, in parallel. Environment-keyed serialisation, the divergence classifier, and comparison against amicontained and seccomp-diff on shared targets. Format frozen at the end of week 9.
Weeks 9-11	Attribution. Confidence model, capability and LSM correlation, and measurement of the undecidable fraction.
Weeks 10-12	Evaluator and remediation, in parallel. Finding categorisation against the manifest and the reference posture, minimal policy emission in both output forms, and closure verification.
Weeks 13-15	Evaluation. Environment matrix, divergence measurement, AF_ALG reproduction, and attribution accuracy against constructed ground truth.
Week 16	Integration buffer. Held deliberately empty. A compressed schedule with no slack fails on its first surprise, and the surprises in this project are kernel-level and hard to predict.
Week 17	Documentation. Final report, demonstration, release.

The system is independently defensible from the end of week 9, at which point it measures, classifies and diffs confinement across environments without evaluating it against a posture or remediating it. That is the point below which the project should not be allowed to fall.

9.2 What the compression gives up

Four reductions make the schedule fit, and each is a deliberate choice rather than an estimate.

The probe corpus is split into a committed set and a deferred set, as Objective 2 records. The committed set is chosen to carry the motivating case and the known filter-bypass paths; the deferred set is specified in the same format so that it is additive work rather than redesign. Corpus breadth is the safest thing to cut, because breadth is the substrate of this project and the classifier, posture and evaluator are its contribution.

The privileged helper is outlined but neither specified nor built. Attribution therefore rests on errno oracles, capability correlation and the process's own security-module label, none of which needs privilege, and the

undecidable fraction is reported rather than reduced.

The environment matrix is reduced to six required cells, with managed clusters as additions, which also removes cloud account and budget risk from the critical path.

SARIF output is deferred behind JSON and human-readable reporting.

Nothing in the measurement, classification or evaluation path is reduced. The instrument does less, and reports what it does with the same rigour.

10. TOOLS AND TECHNOLOGIES

Languages	Rust for the probe engine, chosen for the absence of a managed runtime; Go for the control plane, chosen for the container ecosystem libraries
Crates	nix and libc for direct syscall access, seccompiler for filter construction in test fixtures, serde for fingerprint serialisation
Libraries	client-go for Kubernetes integration, go-github for CI reporting, the Security Profiles Operator API types for custom resource emission
Kernel interfaces	seccomp-BPF, capabilities, namespaces, cgroups v2, procfs and sysfs, io_uring registration, netlink, AppArmor and SELinux label interfaces
Comparison targets	amicontained, seccomp-diff
Development	Disposable virtual machines with snapshot rollback on an amd64 host with hardware virtualisation; Ubuntu 24.04 as the primary target, kernel 6.1 or later

No database, message queue or orchestration layer is required by the instrument itself. The probe image is statically linked and dependency-free by construction, since any library it carries is itself a source of syscalls that pollute the measurement.

11. EVALUATION

11.1 Environment matrix

The unit of evaluation is the environment cell, defined by the tuple of architecture, runtime and version, kernel, distribution and loaded modules, LSM, and RuntimeClass. Cells are counted honestly: kind, minikube and k3s on the same host are one cell, not three, because all three are containerd on the same kernel. Docker Desktop and Colima on macOS, by contrast, run

different guest kernels and distributions and would be two cells, which is why neither is used: cell 1 names its guest explicitly.

Six cells are required. Cell 2 is the baseline, and every other cell moves as few axes as possible relative to it, so that a divergence is attributable.

1. arm64, Ubuntu 24.04 under Lima, containerd 2.2 and runc, AppArmor
2. amd64, Ubuntu 24.04, containerd 2.2 and runc, AppArmor
3. amd64, Fedora or Rocky, containerd 2.2 and runc, SELinux
4. amd64, Ubuntu 24.04, containerd 1.7.12 and runc, AppArmor
5. amd64, Ubuntu 24.04, containerd 2.2, gVisor RuntimeClass
6. amd64, Ubuntu 24.04, containerd 2.2, Kata Containers RuntimeClass

Cells 4, 5 and 6 each move exactly one axis: runtime version, and RuntimeClass twice. Cell 1 moves architecture and, since no distribution builds an identical kernel for both, the kernel build with it; cell 3 moves the LSM and the distribution together, since SELinux is not a supported configuration of Ubuntu. Divergence in those two pairs is attributed to architecture or to policy only after the kernel-version class has been excluded probe by probe. The containerd versions are pinned rather than left to the host's patch state: Ubuntu 24.04 ships 1.7.12 in its release pocket and 2.2 in its updates pocket, and the `io_uring deny` entered the default profile in 2.0, so an unpatched and a patched host of the same release sit on either side of the change. Cell 4 also serves the `AF_ALG` reproduction, whose RuntimeDefault profile must predate any `AF_ALG deny`.

Three further cells are additions, attempted if the schedule permits: an amd64 Podman and crun cell and an amd64 Moby cell, both on the AppArmor host, so that the runtime axis of objective 8 is moved rather than only the runtime version; EKS on Amazon Linux 2023 or GKE on Container-Optimized OS; and Talos, the platform on which the motivating result was published. All six required cells run on developer hardware, which assumes that the team has one amd64 machine with hardware virtualisation: the Kata cell needs it outright, and the other amd64 cells run on an arm64 laptop only under emulation, too slowly to measure runtime cost. That machine is confirmed in week 1. With it, the evaluation depends on neither cloud budget nor account availability, and managed clusters cannot delay the critical path.

11.2 Metrics and success criteria

Divergence across cells, as the number of probes whose verdict differs between cell pairs, reported separately for each divergence class.

Success: policy-explained divergence is measured in at least four cell pairs, and architecture-explained and runtime-class-explained divergence are correctly separated from it in every pair.

Expectation and claim violations, by category and by cell. Success: at least one expectation violation found in at least three distinct cells.

Reproduction of the `AF_ALG` case unaided, demonstrating detection of what previously required bespoke test images. Success: binary.

Attribution accuracy against a constructed ground truth of environments in which each denial has a single known cause. Success: at least 90 per cent of the denials the instrument attributes are attributed correctly, and the fraction it reports as undecidable, measured over all denials, does not exceed 10 per cent.

Remediation closure rate, verified by re-probing. Success: 100 per cent for seccomp-attributed divergences; reported without a threshold for divergences attributed to other mechanisms, which the instrument may identify but cannot always close.

Residual state after a full run, across the five tracked object classes. Success: zero unreclaimed objects. Modules loaded by autoload are reported separately and are not counted, since the container cannot reclaim them.

Runtime cost. Success: full corpus under 60 seconds, reduced CI corpus under 5 seconds.

Comparison against amicontained and seccomp-diff on identical targets, reported as findings produced by each.

12. RISKS

The reference posture is authored by us and is therefore a judgement. Mitigated by requiring a citation for every assertion, by versioning it so that findings are reproducible against a stated posture revision, and by reporting expectation violations separately from claim violations so that a reader can weigh them differently.

Remediation may be undeliverable on managed clusters. A Localhost seccomp profile requires the file to exist at the kubelet's seccomp root on every node, which is not achievable on EKS or GKE without custom node images or a privileged installer. Mitigated by emitting a Security Profiles Operator custom resource as the primary output form, which solves distribution, and by stating that closure verification is performed on self-managed nodes.

The instrument may be indistinguishable from an attack to runtime security tooling. Mitigated by publishing a documented signature for the probe image and by providing a reduced corpus that omits the probes most likely to raise alerts, so that operators can allowlist it deliberately. That corpus necessarily omits the AF_ALG probe, on which the published Copy Fail detection rules key; on a monitored cluster the motivating case is demonstrated by allowlisting the signed image and running the full corpus.

Denial attribution may be partially undecidable. Addressed by the errno oracle as the primary technique, which needs no privilege, and by the confidence model, with the residual fraction reported as a finding. The instrument remains useful with attribution disabled, since reachability is the primary measurement and attribution refines it.

The project may become breadth without depth, producing a larger amicontained rather than an instrument. This is the principal risk. It is mitigated by freezing the corpus at a declared size at the end of week 8 and by treating the diff classifier, the reference posture and the evaluator as the contribution, not the corpus.

A probe may destabilise a host during development. Mitigated by development inside disposable virtual machines with snapshot rollback, and by the side effect declaration requirement, which the engine enforces rather than leaving to convention.

Four-member projects fail through unclear ownership. Addressed by cutting along the fingerprint interface, with one member explicitly responsible for integration.

The schedule is compressed and has correspondingly little tolerance for a foundational slip. The first three weeks are the exposure: every probe family depends on fork isolation and SIGSYS survival, and a defect there is discovered late and cascades. Mitigated by holding the engine's scope fixed and small, by having the other three workstreams develop against the specification format rather than against the engine itself, and by keeping week 16 empty as declared slack. If the engine slips, the deferred corpus set is abandoned before any reduction is made to attribution, evaluation or the environment matrix.

13. ETHICS AND RESPONSIBLE USE

The instrument probes security boundaries, and three commitments follow.

The corpus contains no exploit code. Probes establish that a kernel entry point is reachable and stop there; no probe performs an operation whose success confers privilege, escapes a container, or damages the host. Where an enumeration interface can answer the question without executing the probed operation, it is used in preference.

The instrument is intended for infrastructure the operator owns or has written authorisation to test. This is stated in the documentation and in the tool's own output.

Where the instrument identifies a genuine defect in a runtime, distribution or managed platform rather than a misconfiguration by its operator, the finding is reported to the responsible vendor under coordinated disclosure before publication. Publication means the public release of the instrument and the report; the examined report is submitted regardless, and where a vendor's window has not closed by then it describes the finding generically and withholds the detail until it has.

14. SCOPE

Out of scope: continuous runtime monitoring and intrusion detection, which is the province of existing tooling; generation of seccomp profiles from application workload traces, which is well served and is not this project's question; exploitation of the entry points the instrument measures; userspace package vulnerability scanning; and non-Linux platforms.

The instrument measures and reports. It does not enforce, and it is not a substitute for the enforcement mechanisms whose effect it measures.

15. REFERENCES

All online sources accessed September 2026.

1. MITRE. CVE-2026-31431. Linux kernel algif_aead improper handling of in-place operations.
2. Cybersecurity and Infrastructure Security Agency. Known Exploited Vulnerabilities Catalog, entry for CVE-2026-31431, added 1 May 2026.
3. Microsoft Security Blog. CVE-2026-31431: Copy Fail vulnerability enables Linux root privilege escalation across cloud environments. 1 May 2026.
4. Sysdig. CVE-2026-31431: Copy Fail Linux kernel flaw lets local users gain root in seconds. 2026.
5. Juliet Security. Copy Fail in Kubernetes: RuntimeDefault did not block AF_ALG. 2026.
6. Patel, H., Barrera, D., Somayaji, A. Houdini: Benchmarking Container Security Confinement. Proceedings of the International Conference on Privacy, Security and Trust, 2025.
7. Ghavamnia, S., Palit, T., Mishra, S., Polychronakis, M. Confine: Automated System Call Policy Generation for Container Attack Surface Reduction. RAID, 2020.
8. genuinetools/amicontained. Container introspection tool. Release v0.4.9, November 2019.
github.com/genuinetools/amicontained
9. antitree/seccomp-diff. Extraction and comparison of seccomp-BPF filters from running containers.
github.com/antitree/seccomp-diff
10. kubernetes-sigs/security-profiles-operator.
github.com/kubernetes-sigs/security-profiles-operator
11. containers/oci-seccomp-bpf-hook. OCI hook to trace syscalls and generate a seccomp profile.
github.com/containers/oci-seccomp-bpf-hook

12. `inspektor-gadget`. `advise seccomp-profile` and `advise network-policy`. `inspektor-gadget.io`
13. `moby/moby` pull request 46762. `seccomp: block io_uring_* syscalls` in default profile. Merged November 2023; released in Moby 25.0, January 2024.
14. `containerd` pull request 9320. Update `RuntimeDefault seccomp` profile to disallow `io_uring` related syscalls. Merged November 2023; released in `containerd` 2.0, November 2024.
15. Kubernetes. Pod Security Standards.
`kubernetes.io/docs/concepts/security/pod-security-standards`
16. Open Container Initiative. Runtime Specification, version 1.1.
17. Linux manual pages: `seccomp(2)`, `seccomp_unotify(2)`, `ptrace(2)`, `capabilities(7)`, `socket(2)`, `netlink(7)`, `ioctl(2)`, `bpf(2)`, `io_uring_register(2)`, `keyctl(2)`, `mount(2)`, `prctl(2)`, `perf_event_open(2)`, `userfaultfd(2)`, `open_by_handle_at(2)`, `namespaces(7)`, `user_namespaces(7)`.
18. `moby/moby` pull request 52501. `seccomp: block AF_ALG sockets` in default profile (CVE-2026-31431). Merged May 2026; released in Moby 29.4.2.

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Technology in Computer Science and Engineering.